

Information Retrieval

Risposte alle Domande d'Esame

Raccolta ragionata e deduplicata delle domande dagli Esami A–O,
con risposte estese che collegano i concetti del corso.

Indice

1	Valutazione dei Sistemi di IR	2
2	Term Weighting: TF-IDF, BM25 e Toy Example	5
3	Indici Inversi e Query Processing	9
4	Learning to Rank	13
5	Neural Information Retrieval	18
6	Approximate Nearest Neighbor Search	21
7	Materiale Supplementare (fuori dal syllabus principale)	27

Capitolo 1

Valutazione dei Sistemi di IR

Domanda d'esame

Descrivere gli approcci in letteratura per valutare le prestazioni di un sistema di ricerca (Precision, Recall, F-Measure, MAP, MRR, DCG, NDCG). Perché queste metriche appartengono a filoni diversi della letteratura? Quali le differenze principali? [Esami C, G, H, L; variante “beyond binary relevance”: D, O]

La felicità dell'utente di un motore di ricerca è difficile da misurare direttamente: l'Information Retrieval usa perciò la **rilevanza** dei risultati come proxy principale. Questo paradigma sperimentale nasce dagli esperimenti di Cranfield di Cyril Cleverdon e si è poi standardizzato con la **Text REtrieval Conference** (TREC, NIST), che fornisce l'infrastruttura per test su larga scala. Un benchmark di valutazione è composto da tre elementi: una collezione di documenti, una suite di query e un insieme di giudizi di rilevanza. Poiché giudicare manualmente ogni coppia query-documento in collezioni di milioni di documenti è impraticabile, TREC introduce il **Pooling**: si raccolgono i top risultati di più sistemi (run), si combinano in un pool e solo questi vengono giudicati da assessori umani; i documenti non pooled sono assunti non rilevanti.

A partire da questa base sperimentale, la letteratura ha sviluppato le metriche di valutazione lungo due filoni storicamente distinti, corrispondenti a due modi diversi di intendere la rilevanza.

Il filone classico: rilevanza binaria e insiemi non ordinati. Precision e Recall nascono nel recupero di informazioni “da biblioteca” (basi dati legali, bibliografiche), dove l'obiettivo è spesso la completezza o l'accuratezza pura su un insieme di risultati, senza un ordine particolarmente rilevante:

$$P = \frac{tp}{tp + fp} = \frac{|Rel \cap Ret|}{|Ret|} \quad R = \frac{tp}{tp + fn} = \frac{|Rel \cap Ret|}{|Rel|}$$

La *Precision* è la frazione di documenti recuperati che sono rilevanti; la *Recall* è la frazione di documenti rilevanti totali che sono stati recuperati. Poiché esiste un trade-off intrinseco tra le due (recuperare tutto massimizza la recall ma abbassa la precision, e viceversa), si usa la **F-Measure**, media armonica delle due:

$$F = \frac{2PR}{P + R}, \quad F_\beta = \frac{(1 + \beta^2)PR}{\beta^2 P + R}$$

La media armonica è preferita a quella aritmetica perché resta vicina al minimo delle due componenti, penalizzando fortemente gli sbilanciamenti (con recall=100% e precision molto bassa, F resta vicino alla precision). Con $\beta < 1$ si dà più peso alla precision, con $\beta > 1$ al recall. Precision e Recall, tuttavia, non tengono conto della posizione dei documenti nella lista: un sistema che mette il documento rilevante in prima posizione ottiene lo stesso punteggio di uno

che lo mette in centesima, il che è del tutto contrario all'esperienza reale di un utente di un motore di ricerca.

Verso il rango: MAP e MRR. Con l'affermarsi della ricerca ordinata (ranked retrieval) nascono metriche rank-aware ma ancora a rilevanza binaria. La **Mean Average Precision (MAP)** calcola, per ogni query, l'*Average Precision* (media dei valori di Precision calcolati a ogni posizione in cui compare un documento rilevante), poi ne fa la media aritmetica (macro-averaging) su tutte le query; a differenza della semplice P@K, l'AP tiene conto di tutte le posizioni dei documenti rilevanti, non solo dei primi K. La **Mean Reciprocal Rank (MRR)** nasce invece per gli scenari di *known-item search* o query navigazionali, dove esiste un'unica risposta corretta cercata dall'utente: si calcola il reciproco della posizione del primo documento rilevante trovato, mediato sulle query,

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

riflettendo lo sforzo dell'utente nel raggiungere la risposta.

Il filone moderno: rilevanza graduata e Web Search. Le metriche binarie non catturano le sfumature di utilità tra documenti (un documento può essere parzialmente rilevante, molto rilevante o inutile). Per questo, nella letteratura più recente legata al Web Search e al Learning-to-Rank, si adotta la **rilevanza graduata** (tipicamente su scala 0–4), basata su due assunzioni: i documenti molto rilevanti sono più utili di quelli marginalmente rilevanti, e la loro utilità decresce se appaiono in posizioni basse. Il **Cumulative Gain (CG)** è la metrica più semplice: somma i punteggi di rilevanza rel_i fino al rango p , $CG_p = \sum_{i=1}^p rel_i$, ma non considera l'ordine (scambiare un documento molto rilevante in prima posizione con uno poco rilevante in fondo non cambia CG_p). Il **Discounted Cumulative Gain (DCG)** introduce uno sconto logaritmico sul rango per penalizzare i documenti rilevanti posizionati in basso:

$$DCG_p = rel_1 + \sum_{i=2}^p \frac{rel_i}{\log_2(i)} \quad \text{oppure, in versione "industriale":} \quad DCG_p = \sum_{i=1}^p \frac{2^{rel_i} - 1}{\log_2(i + 1)}$$

la seconda formulazione enfatizza maggiormente i documenti altamente rilevanti tramite il termine esponenziale. Poiché il DCG non è confrontabile fra query diverse (dipende dal numero di documenti rilevanti disponibili per ciascuna), si normalizza rispetto all'**Ideal DCG (IDCG)**, il DCG del miglior ordinamento possibile:

$$NDCG_p = \frac{DCG_p}{IDCG_p} \in [0, 1]$$

con $NDCG_p = 1$ che indica un ordinamento perfetto fino alla posizione p .

Perché filoni diversi. La differenza sostanziale tra le due famiglie è duplice: la **sensibilità alla posizione** (ranking-awareness, assente in Precision/Recall/F-Measure, centrale in MAP/MRR/DCG/NDCG) e la **granularità del giudizio di rilevanza** (binaria per Precision/-Recall/MAP/MRR, graduata per CG/DCG/NDCG). Le prime derivano da un contesto storico in cui l'utente esaminava l'intero insieme dei risultati; le seconde nascono quando ci si rende conto che l'utente reale di un motore di ricerca guarda quasi esclusivamente la prima pagina di risultati (top- k), rendendo cruciale sia l'ordine sia la qualità relativa di ogni singolo documento.

Osservazione: due metriche particolarmente utili nella pratica

Tra tutte, **NDCG** e **MRR** meritano una menzione speciale per la loro diffusione. NDCG è oggi lo standard de facto per valutare motori di ricerca Web e sistemi di Learning-to-Rank, perché combina in un solo numero normalizzato sia la graduazione della rilevanza sia la posizione, ed è la metrica che LambdaMART e simili cercano di ottimizzare indirettamente.

te. MRR resta insostituibile per i task di tipo “known-item” (question answering, ricerca navigazionale, sistemi conversazionali) dove esiste una sola risposta corretta e ciò che conta è quanto in fretta l’utente vi arrivi: è concettualmente più semplice di NDCG ma cattura perfettamente l’esperienza utente in quello scenario specifico.

Capitolo 2

Term Weighting: TF-IDF, BM25 e Toy Example

Domanda d'esame

Descrivere in dettaglio i concetti fondamentali di TF-IDF e BM25. Perché il secondo è considerato un'importante evoluzione del primo? [Esami C, H]

Il modello **TF-IDF** nasce per superare i limiti del Retrieval Booleano (che non produce un ranking) e del coefficiente di Jaccard (che ignora la frequenza dei termini e la loro rarità). Si fonda su due componenti moltiplicate tra loro. La **Term Frequency** (tf) cattura quanto un termine è rappresentativo del contenuto locale di un documento, ma la frequenza grezza non è un buon indicatore diretto: un documento con 10 occorrenze di un termine non è 10 volte più rilevante di uno con una singola occorrenza. Si applica quindi un **log-frequency weighting**:

$$w_{t,d} = \begin{cases} 1 + \log_{10}(tf_{t,d}) & \text{se } tf_{t,d} > 0 \\ 0 & \text{altrimenti} \end{cases}$$

che cresce molto più lentamente all'aumentare delle occorrenze (passando da 1 a 1000 occorrenze il peso passa solo da 1 a 4). L'**Inverse Document Frequency** (idf) cattura invece il potere discriminante globale di un termine nella collezione: termini comuni ("il", "essere") hanno idf vicino a 0, termini rari hanno idf alto,

$$idf_t = \log_{10} \left(\frac{N}{df_t} \right)$$

dove df_t è il numero di documenti che contengono t e N il numero totale di documenti. Lo schema completo è

$$w_{t,d} = (1 + \log(tf_{t,d})) \times \log \left(\frac{N}{df_t} \right)$$

Un punto spesso frainteso è che l'idf da solo, in una query a termine singolo, non induce alcun ordinamento (è identico per tutti i documenti che contengono t): serve sempre in combinazione con il tf locale. Esistono numerose varianti di tf/idf/normalizzazione, identificate dalla **notazione SMART** (es. `lnc.ltc`: tf logaritmico, idf nullo o presente, normalizzazione coseno).

Il TF-IDF, pur efficace, presenta due limiti concreti che il **BM25** (Best Match 25, sviluppato nel sistema Okapi) risolve esplicitamente, ed è per questo considerato la sua evoluzione naturale piuttosto che un modello completamente diverso.

Limite 1 – crescita illimitata del punteggio con tf. Nel TF-IDF il peso logaritmico continua a crescere (seppur lentamente) all'aumentare di tf, senza mai saturare del tutto: un

documento patologicamente ripetitivo può ancora accumulare punteggio. Il BM25 introduce la **saturazione della frequenza**: l'influenza del termine si avvicina asintoticamente a un valore massimo, controllato dal parametro k_1 , oltre il quale ulteriori occorrenze hanno impatto trascurabile.

Limite 2 – gestione grezza della lunghezza del documento. Il TF-IDF, anche con normalizzazione coseno, non calibra esplicitamente l'importanza della lunghezza rispetto alla media della collezione. Il BM25 introduce la **normalizzazione della lunghezza** tramite il parametro $b \in [0, 1]$, confrontando la lunghezza del documento $|d|$ con la lunghezza media $avgdl$: con $b = 0$ nessuna normalizzazione, con $b = 1$ normalizzazione completa. La formula completa (RSV, Retrieval Status Value) è:

$$RSV_d = \sum_{t \in Q} IDF_t \cdot \frac{tf_{t,d} \cdot (k_1 + 1)}{tf_{t,d} + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)}$$

con $k_1 \in [1.2, 2.0]$ e $b = 0.75$ tipici. Grazie a questi due meccanismi, il BM25 rimane oggi la baseline lessicale di riferimento anche nei sistemi neurali ibridi, ed è tuttora lo stadio iniziale (recall-oriented) di molte pipeline di ranking multi-stadio moderne.

Domanda d'esame

Costruire un toy example (5 documenti, 1 query) per introdurre binary term-document incidence matrix, term-document count matrix e term-document frequency matrix. Caratteristiche principali del TF-IDF. Calcolare il TF-IDF sull'esempio. [Esami D, O]

Consideriamo una collezione di 5 documenti con vocabolario ridotto {gatto, cane, topo}: D1 = “il gatto insegue il cane”, D2 = “gatto gatto gatto”, D3 = “il cane dorme”, D4 = “il topo mangia”, D5 = “il gatto e il topo”. Query q = “gatto cane”.

La **binary term-document incidence matrix** registra solo presenza (1) o assenza (0) del termine, senza contarne le occorrenze – è la base del Boolean Retrieval:

Termine	D1	D2	D3	D4	D5
gatto	1	1	0	0	1
cane	1	0	1	0	0
topo	0	0	0	1	1

Non permette il ranking: D1 e D2 risultano identici per “gatto”, anche se D2 ne parla molto di più.

La **term-document count matrix** registra invece le occorrenze grezze (tf), introducendo il Bag of Words:

Termine	D1	D2	D3	D4	D5
gatto	1	3	0	0	1
cane	1	0	1	0	0
topo	0	0	0	1	1

Distingue D2 da D1, ma la frequenza grezza da sola non implica rilevanza proporzionale.

La **term-document frequency matrix** applica il log-frequency weighting $w_{t,d} = 1 + \log_{10}(tf_{t,d})$: D2(gatto) ha $tf = 3 \rightarrow 1 + \log_{10}(3) \approx 1.477$; D1(gatto) ha $tf = 1 \rightarrow 1$.

Termine	D1	D2	D3	D4	D5
gatto	1.0	1.48	0	0	1.0
cane	1.0	0	1.0	0	0
topo	0	0	0	1.0	1.0

“Schiaccia” le frequenze alte per evitare che la ripetizione ossessiva di una parola domini il punteggio.

Calcolo del TF-IDF. Con $N = 5$: gatto appare in D1, D2, D5 ($df = 3$) $\Rightarrow IDF_{gatto} = \log_{10}(5/3) \approx 0.22$; cane appare in D1, D3 ($df = 2$) $\Rightarrow IDF_{cane} = \log_{10}(5/2) \approx 0.40$ (più raro, pesa quasi il doppio). Score TF-IDF per la query “gatto cane”: **D1**: gatto = $1.0 \times 0.22 = 0.22$; cane = $1.0 \times 0.40 = 0.40$; totale = 0.62. **D2**: gatto = $1.48 \times 0.22 \approx 0.33$; cane = 0; totale = 0.33. D1 vince su D2 (0.62 vs 0.33): anche se D2 parla molto di “gatti”, D1 menziona entrambi i termini della query, e il termine “cane” (più raro) spinge in alto il suo punteggio grazie all’IDF – il TF-IDF bilancia sempre frequenza locale e rarità globale.

Domanda d’esame

Data la collezione: D1 “we love information retrieval”; D2 “information retrieval in Pisa is what we need”; D3 “information retrieval course in Pisa”; D4 “passionate about information retrieval”; D5 “we like free candies during the exam”. Fornire le matrici binaria e TF-IDF; per l’IDF scrivere la formula coi numeri corretti (senza calcolare il valore finale). [Esame L]

Il vocabolario della collezione ($N = 5$ documenti) è: we, love, information, retrieval, in, pisa, is, what, need, course, passionate, about, like, free, candies, during, the, exam. Si osservi che, in questo esempio, nessun termine si ripete all’interno dello stesso documento: la **matrice binaria** e la **matrice dei conteggi** coincidono esattamente.

Termine	D1	D2	D3	D4	D5	df_t
we	1	1	0	0	1	3
love	1	0	0	0	0	1
information	1	1	1	1	0	4
retrieval	1	1	1	1	0	4
in	0	1	1	0	0	2
pisa	0	1	1	0	0	2
is	0	1	0	0	0	1
what	0	1	0	0	0	1
need	0	1	0	0	0	1
course	0	0	1	0	0	1
passionate	0	0	0	1	0	1
about	0	0	0	1	0	1
like	0	0	0	0	1	1
free	0	0	0	0	1	1
candies	0	0	0	0	1	1
during	0	0	0	0	1	1
the	0	0	0	0	1	1
exam	0	0	0	0	1	1

Per ciascun termine, l’idf con i numeri corretti è $idf_t = \log_{10}(5/df_t)$, quindi ad esempio $idf_{we} = \log_{10}(5/3)$, $idf_{information} = idf_{retrieval} = \log_{10}(5/4)$, $idf_{in} = idf_{pisa} = \log_{10}(5/2)$, e per tutti i termini con $df_t = 1$ (love, is, what, need, course, passionate, about, like, free, candies, during, the, exam) si ha $idf_t = \log_{10}(5/1) = \log_{10} 5$.

Poiché ogni $tf_{t,d}$ presente vale 1, il peso log-frequency è $w_{t,d} = 1 + \log_{10}(1) = 1$: la **matrice TF-IDF** si ottiene quindi moltiplicando la matrice binaria per l'idf di riga, cioè ogni cella non nulla assume esattamente il valore idf_t del proprio termine (e resta 0 dove il termine è assente) – una diretta conseguenza dell'assenza di ripetizioni interne ai documenti in questo specifico esempio.

Vale la pena notare come "information" e "retrieval" compaiano in 4 documenti su 5: il loro idf è basso ($\log_{10}(5/4)$, vicino a 0), quindi pur essendo i termini più frequenti in valore assoluto contribuiscono poco al punteggio finale – esattamente il comportamento voluto dall'idf, che penalizza i termini troppo diffusi nella collezione a vantaggio di termini più discriminanti come "pisa" o "course".

Capitolo 3

Indici Inversi e Query Processing

Domanda d'esame

Simulare la valutazione della query **Pluto AND NOT Pippo** con la strategia Term-At-A-Time (TAAT), liste invertite Pluto: 2,3,5,9; Pippo: 3,4,5,11,12. [Esami C, H]

La strategia **Term-at-a-time (TAAT)** elabora completamente la posting list di un termine prima di passare al successivo, mantenendo un insieme di **accumulatori** per i documenti candidati.

1. **Inizializzazione:** insieme di accumulatori vuoto.
2. **Processamento di “Pluto”:** si scorre l'intera lista {2, 3, 5, 9}; questi docID diventano i candidati iniziali negli accumulatori.
3. **Processamento di “NOT Pippo”:** si scorre la lista di Pippo {3, 4, 5, 11, 12}; per ogni docID presente, se è negli accumulatori viene rimosso: doc 3 (presente → rimosso), doc 4 (non presente → ignorato), doc 5 (presente → rimosso), doc 11 e 12 (non presenti → ignorati).
4. **Risultato finale:** accumulatori rimasti = {2, 9}.

I documenti che soddisfano la query sono **2** e **9**. Da notare che il TAAT è semplice da implementare (basta `next()` su ogni lista) ma comporta molti cache miss (accumulatori sparsi in memoria) e non permette di saltare docID per query selettive come questa – a differenza del DAAT con `nextGEQ`, che avrebbe potuto saltare direttamente sui docID candidati senza scorrere per intero entrambe le liste.

Domanda d'esame

Simulare la valutazione della query **Pippo AND NOT Pluto** con TAAT, liste Pippo: 1,2,4,8; Pluto: 2,3,4,10,11. [Esame G]

Stesso procedimento della domanda precedente, con termini e liste invertiti:

1. **Inizializzazione:** accumulatori vuoti.
2. **Processamento di “Pippo”:** si scorre {1, 2, 4, 8}, che diventano i candidati iniziali.
3. **Processamento di “NOT Pluto”:** si scorre {2, 3, 4, 10, 11}; doc 2 (presente → rimosso), doc 3 (non presente → ignorato), doc 4 (presente → rimosso), doc 10 e 11 (non presenti → ignorati).

4. **Risultato finale:** accumulatori rimasti = $\{1, 8\}$.

I documenti che soddisfano la query sono **1** e **8**.

Domanda d'esame

Spiegare obiettivo e idee principali dell'algoritmo WAND, con un esempio. [Esame L]

Il **WAND (Weak AND)** è una strategia di **potatura dinamica** (dynamic pruning) per il calcolo esatto del Top- k : il suo obiettivo è restituire esattamente gli stessi k documenti che si otterrebbero valutando ogni documento candidato per intero, ma saltando il calcolo su tutti i documenti che non hanno alcuna possibilità di entrare in classifica, evitando così di “toccare” l'intera posting list di ogni termine (come accadrebbe con un DAAT OR naive).

L'idea centrale è mantenere per ogni termine t della query un **Upper Bound** UB_t : il massimo contributo di punteggio che t può dare a un documento qualsiasi (tipicamente il massimo peso nella sua posting list). Un Min-Heap di dimensione k mantiene i migliori risultati trovati finora; la sua radice è la soglia dinamica θ (il punteggio del k -esimo miglior documento attuale). WAND usa un iteratore per ciascuna posting list, ordina i termini per il loro next docID e seleziona un **pivot**: il primo termine (in ordine di docID crescente) tale per cui la somma degli UB dei termini fino a quel punto supera θ (*threshold check*). Se la somma degli UB dei termini precedenti al pivot non supera θ , nessun documento in quell'intervallo può entrare nel top- k : gli iteratori avanzano con **nextGEQ**, saltando intere porzioni di lista senza calcolarne il punteggio esatto.

Esempio: WAND con quattro termini

Sia $Q = \{\text{language, best, programming, rust}\}$, con $UB(\text{language}) = 1.1$, $UB(\text{best}) = 0.3$, $UB(\text{programming}) = 0.6$, $UB(\text{rust}) = 2.5$, e soglia corrente $\theta = 2.1$ (Top-1 attuale: documento 8, score 2.1). Ordinando i termini per next docID e scegliendo come pivot il termine “rust” (il cui next docID è 15), si verifica: $\theta = 2.1 <? UB(\text{language}) + UB(\text{best}) + UB(\text{programming}) + UB(\text{rust}) = 1.1 + 0.3 + 0.6 + 2.5 = 4.5$. La condizione è soddisfatta (4.5 supera 2.1): il documento 15 *può* potenzialmente entrare in classifica e va quindi valutato per intero; se invece la somma degli UB non avesse superato θ , l'intero blocco di documenti fino al pivot sarebbe stato saltato senza ulteriori calcoli.

WAND è particolarmente efficace per query lunghe o disgiuntive (OR), perché agisce da filtro che promuove solo i documenti con una “coincidenza sufficiente” di termini pesanti, riducendo drasticamente il numero di posting effettivamente decomprese e valutate rispetto a un DAAT OR esaustivo.

Exact Top-k Retrieval: WAND

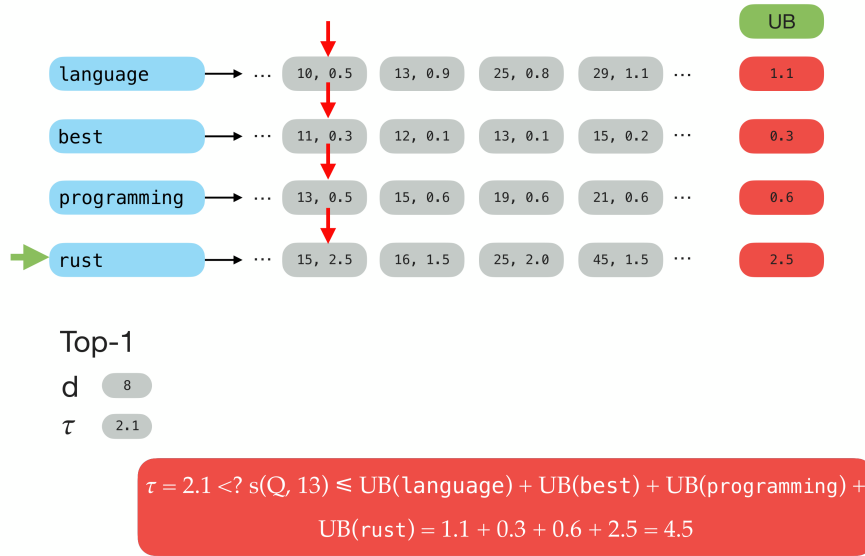


Figura 3.1: Selezione del pivot e threshold check in WAND: si scelgono i termini fino al pivot e si confronta la somma dei loro Upper Bound con la soglia θ .

Domanda d'esame

Codificare i numeri 9, 11, 14 con Elias Gamma ed Elias Delta. Perché un codice per interi deve essere prefix-free? Vantaggi/svantaggi del Variable Byte rispetto a Elias Gamma/Delta. [Esami C, D, G, H, O]

Elias Gamma codifica x scrivendo la sua lunghezza binaria (meno un bit) in unario, seguita dal valore binario di x troncato del bit più significativo (sempre 1, quindi ridondante). **Elias Delta** migliora la compressione per numeri grandi codificando la componente lunghezza stessa con Elias Gamma invece che in unario.

Esempio: codifica di $x = 9$

$9 = 1001_2$ (4 bit). **Gamma**: bit meno uno $N = 3 \rightarrow$ prefisso unario 0001; suffisso (bit di x esclusa la testa) 001. Risultato $\gamma(9) = 0001001$ (7 bit). **Delta**: $L = 1 + \lfloor \log_2 9 \rfloor = 4$; si codifica $L = 4$ in gamma ($4 = 100_2$, bit meno uno = 2 $\rightarrow$ unario 001, suffisso 00, $\gamma(4) = 00100$); si appendono i bit di 9 esclusa la testa: 001. Risultato $\delta(9) = 00100001$ (8 bit).

Esempio: codifica di $x = 11$

$11 = 1011_2$ (4 bit). **Gamma**: $N = 3 \rightarrow$ prefisso 0001; suffisso 011. Risultato $\gamma(11) = 0001011$ (7 bit). **Delta**: $L = 1 + \lfloor \log_2 11 \rfloor = 4 \rightarrow \gamma(4) = 00100$; suffisso di 11 esclusa la testa: 011. Risultato $\delta(11) = 00100011$ (8 bit).

Esempio: codifica di $x = 14$

$14 = 1110_2$ (4 bit). **Gamma**: $N = 3 \rightarrow$ prefisso 0001; suffisso 110. Risultato $\gamma(14) = 0001110$ (7 bit). **Delta**: $L = 1 + \lfloor \log_2 14 \rfloor = 4 \rightarrow \gamma(4) = 00100$; suffisso di 14 esclusa la testa: 110. Risultato $\delta(14) = 00100110$ (8 bit).

Un codice per interi deve essere **prefix-free** (nessuna parola di codice è prefisso di un'altra) perché è l'unico modo per garantire la **decodificabilità univoca** in un flusso continuo di bit, come una posting list compressa. Se un codice fosse prefisso di un altro, il decodificatore non saprebbe se fermarsi o continuare a leggere per identificare un numero più lungo, rendendo impossibile separare correttamente i valori consecutivi: un flusso come 0111010111011101001, codificato con un codice non prefix-free, può essere decodificato in più modi diversi e incompatibili tra loro (ambiguità), rendendo il codice inutilizzabile in pratica.

Il **Variable Byte** (VByte) lavora allineato al byte (7 bit di payload + 1 bit di controllo per byte) anziché al singolo bit. È molto più veloce da decodificare, perché sfrutta operazioni native della CPU allineate al byte, evitando manipolazioni bit-a-bit costose in cicli di clock – un vantaggio decisivo su hardware moderno. Di contro, spreca più spazio per numeri piccoli: usa sempre almeno 8 bit (di cui uno di controllo), mentre Elias Gamma può rappresentare piccoli interi con pochissimi bit; per numeri grandi, Elias Delta comprime generalmente meglio del VByte. La scelta tra i due è quindi un classico trade-off velocità di decodifica contro densità di compressione.

Domanda d'esame

Descrivere la struttura dati Elias-Fano per le posting list. Perché è molto efficiente nel processare le query? [*Esame L*]

Elias-Fano è uno schema di compressione per sequenze di interi ordinati che si avvicina al limite teorico inferiore di compressione (Combinatorial Lower Bound, $\lceil \log_2 \binom{U}{n} \rceil$ bit per rappresentare n elementi scelti da un universo di dimensione U), pur mantenendo – a differenza dei semplici d-gap compressi con Elias/VByte – la possibilità di **accesso casuale** (random access) diretto a un qualunque elemento della sequenza senza doverla decomprimere per intero.

La struttura divide ogni numero ordinato x in due parti: i bit bassi (*low bits*), memorizzati esplicitamente e senza compressione in un array L (tipicamente $\log_2(U/n)$ bit per elemento); e i bit alti (*high bits*), rappresentati in modo implicito ed estremamente compatto come un array di bit H costruito concatenando, per ciascun possibile valore di prefisso alto, una rappresentazione unaria del numero di elementi della sequenza che condividono quel prefisso. Lo spazio complessivo risultante è circa $n \log_2(U/n) + 2n$ bit, molto vicino al limite teorico.

L'efficienza nel processare le query deriva da due proprietà congiunte: (1) l'**accesso diretto** $Access(i)$ a un elemento in tempo $O(1)$, che permette ad esempio in DAAT/WAND di saltare direttamente a una posizione senza scorrere sequenzialmente i d-gap intermedi; (2) l'operazione **NextGEQ**(x) (fondamentale per intersezioni AND e per gli algoritmi di potatura dinamica come WAND) risolvibile in tempo $O(\log(U/n))$ sfruttando le operazioni di **Rank** e **Select** sull'array H (implementabili in tempo costante $O(1)$ con strutture ausiliarie a blocchi e superblocchi), decomprimendo solo pochi valori anziché l'intera lista. Il **Partitioned Elias-Fano** spinge oltre questa idea dividendo la lista in blocchi ed applicando Elias-Fano indipendentemente su ciascuno, ottimizzando localmente il numero di bit bassi e migliorando ulteriormente le prestazioni su distribuzioni non uniformi (tipiche delle posting list reali, che alternano zone dense a zone rare).

Capitolo 4

Learning to Rank

Domanda d'esame

Descrivere un framework generale di learning-to-rank per il Web search. Quali i problemi nell'apprendere il ranking? Descrivere in dettaglio gli approcci Pointwise, Pairwise e Listwise, includendo come si applica la discesa del gradiente al Pointwise, e le caratteristiche delle tre famiglie. *[Esami A, B, E, I, M, N – domanda master che unifica tutte le varianti]*

Il **Learning-to-Rank** (LtR) applica il Machine Learning supervisionato alla costruzione di modelli di ranking nei motori di ricerca moderni. A differenza dei modelli classici (VSM, BM25), dove la funzione di punteggio è una formula analitica con pochi parametri liberi fissati a mano, nel LtR la funzione $f(q, d)$ viene appresa automaticamente dai dati. Il sistema si addestra su un **training set** di triple (q, d, y) : query, documento ed etichetta di rilevanza y (ground truth binaria o graduata, es. scala 0–4), ottenuta da giudizi umani o segnali impliciti (click). Per ogni coppia query-documento si estrae un **vettore di feature** che può includere segnali *query-independent* (PageRank, lunghezza del documento, freschezza), *query-dependent* (score BM25/TF-IDF, posizione della prima occorrenza) e *query-only* (lunghezza della query, intento). L'obiettivo del training è minimizzare una loss function sulle predizioni rispetto alle etichette, producendo un modello capace di ordinare documenti mai visti per query nuove.

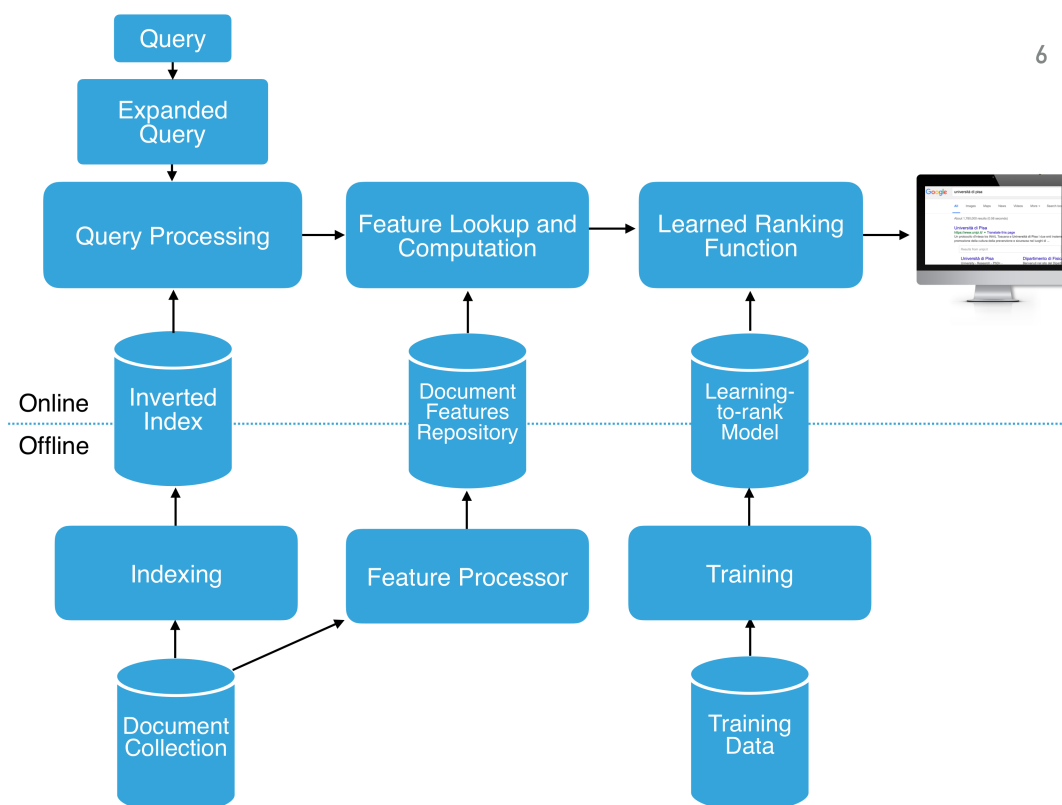


Figura 4.1: Architettura di un motore di ricerca con LtR: la fase offline addestra la Learned Ranking Function sui training data, la fase online la applica alle feature calcolate per i candidati recuperati dall’indice invertito.

Perché apprendere il ranking è difficile. A differenza di classificazione/regressione standard, l’obiettivo del ranking è l’ordine *relativo* dei documenti, ma le metriche di valutazione (NDCG, MAP) sono funzioni a gradino, discontinue rispetto ai parametri del modello – non direttamente differenziabili e quindi difficili da ottimizzare con discesa del gradiente diretta (il gradiente di un’operazione di **sort** è ovunque nullo o indefinito). A questo si aggiungono lo sbilanciamento dei dati e la scarsità di etichette di qualità per la “coda lunga” delle query rare. **LambdaMART** aggira la non differenziabilità definendo dei “gradienti virtuali” (lambda): pesano quanto un documento dovrebbe muoversi nella lista per migliorare la metrica complessiva, senza richiedere che questa sia derivabile in forma chiusa – di fatto un’approssimazione empirica del gradiente di NDCG.

All’interno di questo framework si distinguono tre famiglie di approcci, che differiscono per input considerato e struttura della loss.

Pointwise. Riduce il ranking a un problema di regressione o classificazione classica: l’input è una singola coppia query-documento, l’output un punteggio assoluto (es. probabilità di rilevanza), ignorando completamente la posizione relativa degli altri documenti. Trattando il problema come regressione, la loss tipica è l’errore quadratico medio $L = \sum_i (f(x_i) - y_i)^2$, dove $f(x_i)$ è il punteggio predetto per il vettore di feature x_i e y_i l’etichetta reale. La **discesa del gradiente** aggiorna i parametri θ nella direzione opposta al gradiente della loss: $\theta_{new} = \theta_{old} - \eta \nabla L(\theta)$. Poiché ogni istanza è indipendente dalle altre, il gradiente si calcola facilmente e separatamente su ogni punto del training set – a differenza di Pairwise e Listwise, dove la loss dipende da relazioni tra più documenti contemporaneamente, rendendo l’applicazione diretta della discesa del gradiente più complessa. Il limite del Pointwise è che non considera l’ordine relativo: predire 4.1 invece di 4.0 per il documento più rilevante pesa come lo stesso errore commesso sul centesimo

documento, sebbene l'impatto sul ranking visibile sia nullo nel secondo caso.

Pairwise. Trasforma il problema in classificazione binaria su coppie (d_i, d_j) per la stessa query: l'obiettivo non è predire il punteggio esatto ma minimizzare il numero di **inversioni** (documento meno rilevante posizionato sopra uno più rilevante), garantendo che se d_i è più rilevante di d_j allora $f(d_i) > f(d_j)$. Esempi tipici sono *RankNet* e *LambdaRank*. È più efficace del Pointwise perché cattura la relazione relativa tra documenti, ma può soffrire se non pesato correttamente: correggere un'inversione in fondo alla lista vale, nella loss non pesata, quanto correggerne una in cima (dove invece l'utente osserva quasi esclusivamente i risultati).

Listwise. Opera sull'intera lista di documenti associata a una query, cercando di ottimizzare direttamente (o tramite proxy) le metriche di ranking come NDCG o MAP. È l'approccio più complesso ma teoricamente più accurato, perché cattura nativamente la natura posizionale del problema. Poiché queste metriche sono spesso non differenziabili, si usano approssimazioni o bounding (LambdaMART, ListNet, SoftRank) – rappresenta oggi lo stato dell'arte per i sistemi di ranking basati su alberi.

Osservazione: la famiglia più usata in pratica – Pairwise/Listwise ibridi

Nella pratica industriale, la famiglia più diffusa è quella **Pairwise/Listwise ibrida** rappresentata da LambdaMART: usa una loss pairwise (RankNet, basata sul confronto tra coppie di documenti) ma pesa ogni coppia con un fattore $|\Delta NDCG|$ che stima quanto cambierebbe l'NDCG scambiando quella coppia di posizioni – unendo così la trattabilità computazionale del Pairwise (i gradienti si calcolano su coppie, non sull'intera lista) con la sensibilità alla metrica del Listwise. È il motivo per cui LambdaMART resta oggi uno dei modelli più usati per il ranking finale nelle pipeline multi-stadio dei principali motori di ricerca commerciali.

Domanda d'esame

Descrivere la strategia Gradient-based One-Side Sampling (GOSS) e spiegare perché riduce il tempo di training, con esempi. [Esami A, I]

GOSS è una strategia introdotta in **LightGBM** per accelerare l'addestramento di Gradient Boosted Decision Trees. L'osservazione di partenza è che le istanze con gradiente piccolo sono già ben apprese dal modello corrente (il loro errore residuo è basso) e contribuiscono poco all'information gain di un nuovo split, mentre le istanze con gradiente grande necessitano ancora attenzione. GOSS mantiene *tutte* le istanze con gradiente grande (il top $a \times 100\%$) e campiona casualmente solo una piccola porzione di quelle con gradiente piccolo (il top $b \times 100\%$ dei rimanenti). Per non alterare la distribuzione originale dei dati e non introdurre bias, i gradienti delle istanze campionate a gradiente piccolo vengono amplificati per un fattore $(1 - a)/b$ nel calcolo dell'information gain, compensando così la loro sotto-rappresentazione nel campione.

Questo riduce il tempo di training perché la costruzione degli istogrammi (l'operazione dominante nel costo di ogni split, proporzionale al numero di istanze processate) opera su una frazione $a + b$ del dataset originale invece che sull'intero N , mentre l'accuratezza resta paragonabile al training completo poiché le istanze “facili” contribuiscono poco al guadagno informativo. Un esempio concreto: nel ranking di milioni di pagine Web, anziché iterare su ogni singola coppia query-documento a ogni iterazione di boosting, GOSS scarta la maggior parte delle coppie la cui rilevanza è già predetta con precisione (gradiente piccolo, es. documenti chiaramente irrilevanti o chiaramente in cima), concentrando il calcolo sulle coppie ancora “confuse” e accelerando così la convergenza del modello senza un impatto significativo su NDCG finale.

Domanda d'esame

Descrivere la strategia Exclusive Feature Bundling (EFB) e spiegare perché riduce il tempo di training, con esempi. [Esami B, M]

EFB, anch'essa introdotta in LightGBM, attacca il problema dell'efficienza da un angolo complementare a GOSS: invece di ridurre il numero di istanze N , riduce il numero di **feature** M effettivamente processate. In dataset ad alta dimensionalità e sparsi (tipici, ad esempio, dopo un one-hot encoding, o nelle rappresentazioni sparse apprese usate in IR), molte feature sono **mutuamente esclusive**: non assumono mai valori non nulli contemporaneamente per la stessa istanza. EFB le raggruppa (*bundle*) in un'unica feature densa sintetica, trattando il problema come una colorazione di grafo (le feature sono nodi, la co-occorrenza di valori non nulli è un arco/conflitto): feature con pochi conflitti tra loro vengono unite nello stesso bundle (trovare il numero minimo di bundle è NP-hard, quindi si usa un'euristica greedy).

Per non perdere informazione nel processo, l'algoritmo assegna un **offset** ai valori delle feature originali all'interno del bundle, così che i loro intervalli di valori non si sovrappongano mai: ad esempio, con Feature A in $[0, 10)$ e Feature B in $[0, 20)$, mutuamente esclusive, si può creare un bundle dove A mantiene i valori $[0, 10)$ e B viene rimappata su $[10, 30)$ tramite un offset di 10; un unico valore osservato nel bundle risale così univocamente al valore corretto della feature originale, senza perdita di informazione (mapping lossless).

Questo riduce il tempo di training perché la complessità della costruzione degli istogrammi passa da $O(\text{data} \times \text{feature})$ a $O(\text{data} \times \text{bundle})$, con il numero di bundle tipicamente molto inferiore al numero di feature originali: la velocità di calcolo aumenta proporzionalmente. GOSS ed EFB sono complementari e vengono applicati insieme in LightGBM: il primo riduce N , il secondo M , attaccando entrambi i fattori che rendono costosa la ricerca esaustiva dello split ottimale.

Domanda d'esame

Perché il calcolo dello split ottimale in un albero di decisione è computazionalmente costoso? Quali i principali colli di bottiglia? Discutere almeno 3 approcci per migliorarne l'efficienza.
[Esami E, N]

Il calcolo dello split ottimale è l'operazione più onerosa del training di un albero (o di un ensemble come GBRT) perché richiede una ricerca esaustiva greedy: per ogni nodo, occorre trovare la coppia (f, t) (feature f , soglia t) che massimizza l'information gain (o minimizza la varianza dell'errore). Con N istanze e M feature, l'algoritmo *pre-sorted* standard (Exact Greedy) richiede di ordinare le N istanze per ciascuna delle M feature per poter valutare tutti gli $N - 1$ punti di split candidati, con un costo complessivo di $O(M \cdot N \log N)$ per livello dell'albero.

I principali colli di bottiglia sono tre. Il **costo di ordinamento** stesso: mantenere o ricalcolare l'ordinamento a ogni nodo consuma enormi cicli CPU. I **cache miss**: l'accesso ai gradienti corrispondenti agli indici ordinati delle feature spesso non è contiguo in memoria (le feature sono in columnar storage, ma i gradienti sono indicizzati per riga/documento), causando accessi casuali che saturano la cache. La **frammentazione dei dati**: man mano che l'albero cresce e i nodi si dividono, il numero di istanze per foglia diminuisce e i dati si frammentano in memoria, riducendo l'efficacia di SIMD e prefetching.

Tre approcci per mitigare questi costi, tutti presenti in LightGBM/XGBoost:

- 1. Histogram-based Split Finding.** Invece di valutare tutti gli N valori continui di una feature, questi vengono discretizzati in un numero fisso di bin $K \ll N$ (tipicamente 255) secondo percentili o uniformemente. Si costruisce un istogramma che aggrega i gradienti per bin, e lo split si cerca solo sui confini dei bin, riducendo la complessità da $O(\text{data} \times \text{feature})$ a $O(\text{bin} \times \text{feature})$; gli istogrammi, essendo compatti, stanno in cache L1/L2, eliminando quasi del tutto i cache miss.
- 2. GOSS.** Riduce il numero di istanze N effettivamente processate, mantenendo tutte quelle a gradiente grande e campionando quelle a gradiente piccolo (dettagli sopra), attaccando così direttamente la dimensione N della complessità.
- 3. EFB.** Riduce il numero di feature M raggruppando feature mutuamente esclusive in bundle densi, attaccando la dimensione M della complessità (dettagli sopra).

Le tre tecniche sono complementari: agiscono rispettivamente sul numero di valori da valutare per feature (Histogram), sul numero di istanze N (GOSS) e sul numero di feature M (EFB), ed è per questo che LightGBM le combina tutte per ottenere accelerazioni sostanziali senza una perdita significativa di accuratezza.

Capitolo 5

Neural Information Retrieval

Domanda d'esame

Illustrare l'architettura generale di un query processor basato su metodi representation-based. Differenze tra dense single-vector, dense multi-vector e sparse representation, con pro e contro. [Esami A, E, I]

L'architettura generale dei **Representation-Based Methods** (noti anche come *Bi-Encoder* o *Dual Encoder*) disaccoppia nettamente l'elaborazione del documento da quella della query. I documenti vengono codificati **offline** da un **Document Encoder** (una rete neurale, tipicamente basata su Transformer) in vettori memorizzati in un indice vettoriale; a tempo di query, un **Query Encoder** (spesso identico o speculare) codifica la richiesta **online**, in tempo reale. Il recupero avviene calcolando la similarità geometrica (prodotto scalare o coseno) tra il vettore di query e quelli indicizzati, tramite ricerca **Approximate Nearest Neighbor** (ANN) per gestire efficientemente collezioni di milioni/miliardi di documenti.

Le rappresentazioni si dividono in tre famiglie che differiscono per granularità.

Dense Single-Vector (es. DPR, ANCE): l'intero contenuto semantico di un documento è compresso in un unico vettore denso a dimensione fissa (es. 768), tipicamente dall'embedding del token [CLS] o dal mean pooling. **Pro:** massima efficienza in memoria e latenza, ricerca NN rapidissima, un solo vettore per documento da confrontare. **Contro:** soffre di *Information Bottleneck* – comprimere un passaggio lungo in un solo vettore perde informazioni fini, rendendo difficile il matching preciso su termini specifici o documenti multi-argomento. **ANCE** in particolare migliora il training di questi modelli usando *hard negative* recuperati dinamicamente dall'indice durante il training stesso, correggendo la discrepanza tra la distribuzione vista in training e quella incontrata in inferenza.

Dense Multi-Vector (reso celebre da ColBERT): il documento è rappresentato come una sequenza di vettori, uno per token, non un singolo punto. La similarità si calcola con **Late Interaction**: per ogni token di query si cerca il token di documento con similarità massima (*MaxSim*), poi si sommano questi massimi su tutti i token di query. **Pro:** massima efficacia tra i metodi representation-based, si avvicina alle prestazioni dei costosi Cross-Encoder pur restando indicizzabile offline (gli embedding dei documenti si precalcolano e si inseriscono in un indice ANN). **Contro:** costi di spazio proibitivi (centinaia di vettori per documento, indice fino a due ordini di grandezza più grande) e latenza di ricerca superiore, dovendo confrontare matrici invece di singoli vettori.

Sparse Learned Representation (es. SPLADE): proietta il testo in vettori sparsi ad altissima dimensionalità (pari al vocabolario, es. 30.000), dove i pesi non sono frequenze grezze come in TF-IDF ma valori appresi da un modello neurale (tipicamente l'output MLM di BERT), che

può anche espandere il documento con termini semanticamente correlati ma assenti dal testo originale, tramite regolarizzazione L_1 per mantenere la sparsità. **Pro:** interpretabile (si possono leggere i termini “accesi”), compatibile con l’inverted index tradizionale (maturo e ottimizzato), ottima generalizzazione zero-shot. **Contro:** l’espansione può allungare le posting list rispetto a BM25 puro, richiedendo tecniche di pruning aggressive (es. Seismic); non cattura relazioni posizionali complesse come i modelli multi-vettore.

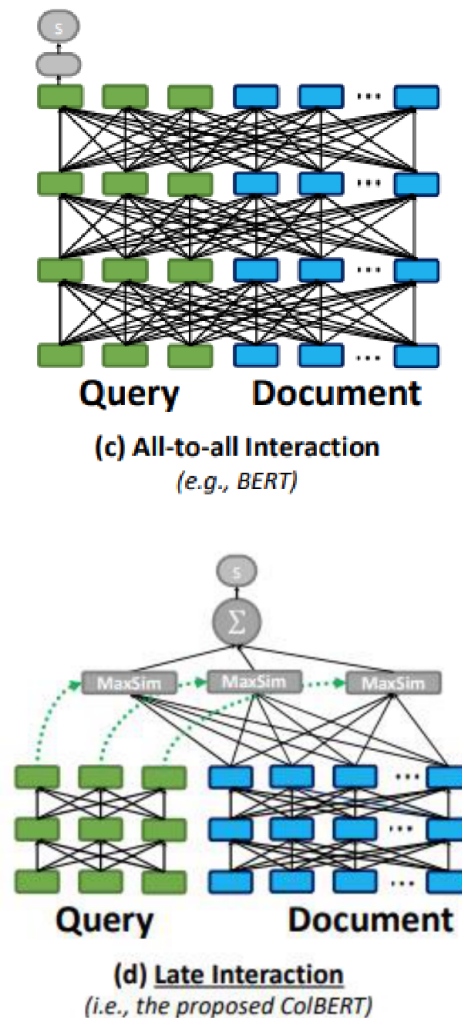


Figura 5.1: All-to-all Interaction (Cross-Encoder) vs Late Interaction (ColBERT, dense multi-vector): quest’ultima calcola MaxSim tra token di query e documento, poi somma i massimi.

Domanda d’esame

Descrivere le idee principali di Word2Vec (Mikolov et al., 2013): task risolti e risultati scientifici. Come si lega il concetto di embedding a questo lavoro? [Esami B, M]

Word2Vec, introdotto da Mikolov et al. nel 2013, è una famiglia di architetture neurali leggere per apprendere rappresentazioni vettoriali dense delle parole (embedding) in modo efficiente su enormi quantità di testo non annotato, basandosi sull’**ipotesi distribuzionale**: il significato di una parola è determinato dal contesto in cui appare. Propone due architetture speculari: **CBOW** (Continuous Bag-of-Words), che predice la parola target a partire dalla media dei vettori delle parole di contesto circostanti (veloce, buone prestazioni su termini frequenti), e **Skip-gram**, che predice le parole di contesto data la parola target (più costoso, ma gestisce

meglio i termini rari). Entrambe sono implementate come reti neurali a due soli livelli lineari, dove gli embedding emergono come sottoprodotto del task di predizione: nonostante sia un task supervisionato nella forma, non richiede alcuna etichettatura manuale, potendo essere addestrato su qualunque corpus di testo grezzo.

Il principale risultato scientifico di Word2Vec è la scoperta che le relazioni semantiche e sintattiche vengono codificate come **operazioni algebriche lineari** nello spazio degli embedding: l'esempio celebre $\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$ dimostra che concetti come il genere sono rappresentati come direzioni consistenti nello spazio vettoriale, indipendentemente dalla parola specifica considerata. Questo risultato ha mostrato per la prima volta su larga scala che rappresentazioni dense apprese in modo non supervisionato catturano struttura linguistica genuina, superando i limiti delle rappresentazioni sparse (one-hot encoding), incapaci per costruzione di misurare alcuna similarità tra termini distinti (tutti i vettori one-hot sono ortogonali fra loro per definizione).

Il concetto di **embedding** nasce qui in senso moderno: ogni parola viene “immersa” (embedded) in uno spazio vettoriale continuo e a bassa dimensionalità (tipicamente 200–300 dimensioni, contro le decine di migliaia del vocabolario originale), dove la vicinanza geometrica corrisponde a similarità semantica. Word2Vec ha reso popolare l'idea che queste rappresentazioni dense potessero fungere da input generico e riutilizzabile per compiti a valle – classificazione, retrieval, traduzione automatica – aprendo la strada a tutta la successiva generazione di modelli neurali per il linguaggio, inclusi FastText (che estende Word2Vec con n-grammi di caratteri per gestire parole fuori vocabolario) e, in ultima analisi, i Transformer ed embedding contestuali come BERT, che a differenza di Word2Vec producono un vettore diverso per la stessa parola a seconda della frase in cui compare.

Capitolo 6

Approximate Nearest Neighbor Search

Domanda d'esame

Rispondere alle seguenti domande sul k-nearest neighbors retrieval: ricerca efficiente in 1D e 2D con distanza euclidea; Product Quantization e calcolo della distanza (con esempio); perché HNSW introduce una gerarchia, con esempio motivante. [Esami A, B, E, I, M, N – domanda master unificata]

La **Similarity Search** cerca, dato un vettore query $q \in \mathbb{R}^d$, il vettore (o i k vettori) dell'insieme $\mathcal{X}$ che minimizzano una distanza (euclidea, o l'inverso della cosine similarity). La difficoltà del problema cresce drasticamente con la dimensionalità.

Caso 1D. Se i punti sono pre-ordinati in un array (costo di indicizzazione $O(n \log n)$), il nearest neighbor si trova con una semplice **Binary Search**: si individua la posizione teorica in cui la query si inserirebbe nell'array ordinato, e il candidato più vicino è necessariamente uno dei due punti adiacenti a quella posizione (predecessore o successore) – basta confrontare le due distanze. Complessità: $O(\log n)$.

Caso 2D (e N-D). Non esiste un ordinamento totale che preservi la località spaziale in più dimensioni (*curse of dimensionality*): non basta più una ricerca binaria su un array lineare. Si usano strutture di **partizionamento spaziale** come il **KD-Tree**, che divide ricorsivamente lo spazio alternando gli assi di taglio (X, poi Y, poi di nuovo X, ...) alla mediana dei punti correnti. Durante la ricerca, se la distanza minima tra la query e il confine di una regione supera la distanza dal miglior candidato trovato finora, l'intero ramo dell'albero viene scartato (pruning), evitando di visitarlo. Un'alternativa concettuale sono i **Diagrammi di Voronoi**, che partizionano il piano in celle (ciascuna contenente i punti più vicini a un dato sito), trasformando la ricerca del vicino più prossimo in un problema di localizzazione del punto in un poligono. Entrambe le strutture offrono in media $O(\log n)$, ma con d crescente (es. $d > 100$, tipico di embedding testuali o visuali) degenerano verso la scansione lineare $O(N)$: per questo, in alta dimensionalità, si accetta una perdita di accuratezza (**Approximate NN**) in cambio di un miglioramento drastico in tempo e memoria, tramite famiglie di tecniche come alberi randomizzati (ANNOY), clustering (IVF), hashing (LSH) o grafi di prossimità (HNSW).

HNSW e la gerarchia. L'algoritmo **Hierarchical Navigable Small World** parte dal grafo NSW (Navigable Small World): ogni vettore è un nodo collegato ai suoi vicini, e grazie a collegamenti a lungo raggio oltre a quelli locali (proprietà "small world") si naviga il grafo greedily avvicinandosi rapidamente alla destinazione. Il problema di un grafo NSW piatto è che la ricerca greedy può bloccarsi in un **minimo locale**: HNSW risolve questo introducendo una **gerarchia di livelli** ispirata alle Skip List di Pugh, dove i livelli superiori sono sparsi con collegamenti molto lunghi (grandi salti nello spazio) e i livelli inferiori sono via via più densi,

fino al livello base (livello 0) che contiene tutti i punti. La ricerca parte dal livello più alto con una ricerca greedy del nodo più vicino a q , poi “scende” al livello sottostante usando quel nodo come punto d’ingresso, raffinando progressivamente fino al livello base, dove si ottengono i vicini finali.

Esempio: analogia stradale per la gerarchia HNSW

Per andare da una piccola via nella città A a una piccola via nella città B, non si percorrono solo strade locali: ci si sposta prima su autostrade (livelli superiori del grafo) per coprire rapidamente grandi distanze e avvicinarsi alla destinazione generale; solo una volta in prossimità della città B si scende verso strade provinciali e infine cittadine (livelli inferiori) per individuare l’indirizzo esatto. In HNSW questo permette di saltare la maggior parte dei punti non rilevanti nei livelli alti e di rifinire la ricerca solo quando si è già nella zona corretta dello spazio, garantendo tempi sub-lineari anche su dataset di scala massiva – è esattamente il motivo per cui la gerarchia viene introdotta: senza di essa, un singolo grafo NSW piatto richiederebbe molti più passi greedy (e rischi di minimo locale) per attraversare grandi distanze nello spazio vettoriale.

Approximate k-NN Retrieval

- Possible strategies are:
 - Tree-based solutions. e.g., ANNOY by Spotify
 - Clustering-based solutions. e.g., IVF in FAISS by Facebook
 - Hashing-based solutions. e.g., LSH from theoretical papers
 - Proximity Graphs. e.g., HNSW by everybody right now

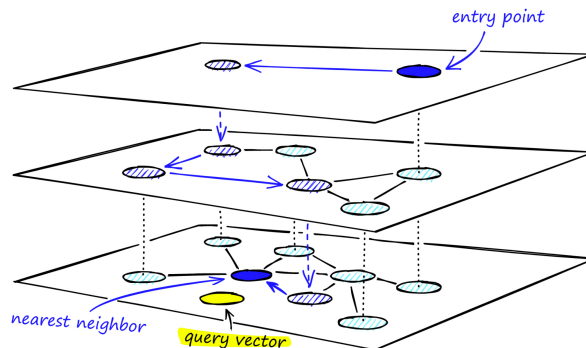


Figura 6.1: Gerarchia di HNSW: la ricerca parte dall’entry point nel livello più alto e scende verso il livello 0, sempre più denso, fino al nearest neighbor.

Product Quantization. Per gestire dataset che non entrano in RAM, o per accelerare il calcolo delle distanze, la **Product Quantization (PQ)** decompone in modo *lossy* un vettore $x \in \mathbb{R}^D$ in m sottovettori di dimensione ridotta $D^* = D/m$. Per ogni sottospazio si esegue un clustering (K-means) sui dati di training, creando un **codebook** indipendente con K centroidi (spesso $K = 256$, per usare esattamente 1 byte per indice); ogni sottovettore viene poi sostituito dall’ID del centroide più vicino nel proprio codebook. Il vettore compresso è quindi la sequenza di m interi, occupando molto meno spazio dei float originali (una codifica *lossy*: vettori diversi possono ricevere la stessa codifica).

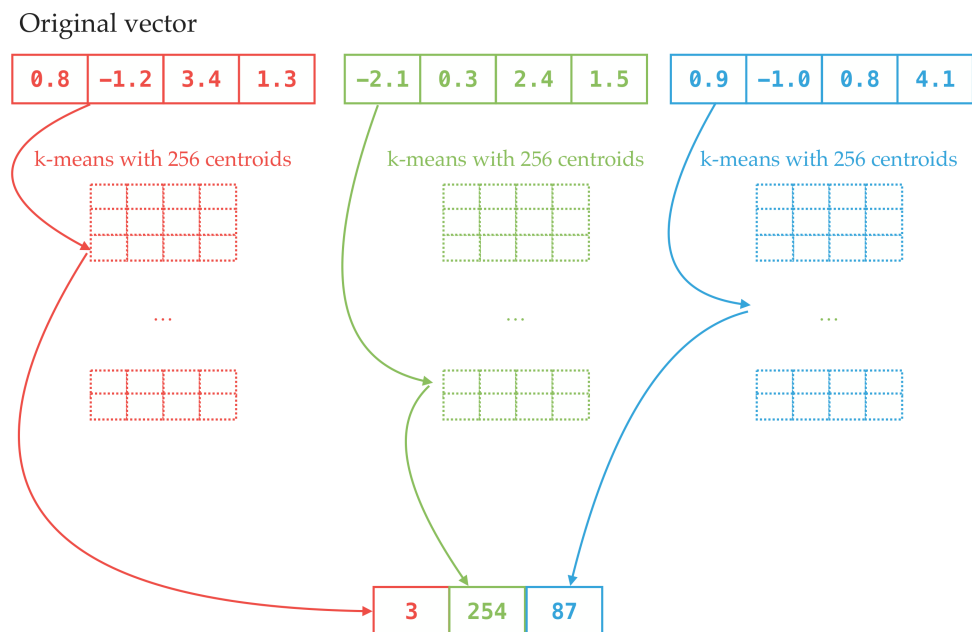
Il calcolo della distanza avviene tramite **Asymmetric Distance Computation (ADC)**: durante la ricerca non si ricostruiscono i vettori originali dai codici. La query q (non quantizzata, per massima precisione) viene divisa negli stessi m sottovettori; per ciascuno si calcola la di-

stanza verso *tutti* i K centroidi del codebook relativo, salvando i risultati in una **look-up table (LUT)**. La distanza approssimata tra q e un documento codificato è semplicemente la **somma** dei valori letti dalla tabella agli indici specificati dalla codifica del documento, eliminando operazioni in virgola mobile costose sui vettori originali.

Esempio: PQ su un vettore a 128 dimensioni

Dividendo un vettore di 128 dimensioni in $m = 8$ segmenti da 16 dimensioni ciascuno, la distanza finale tra query e documento è la somma di 8 distanze parziali, ciascuna recuperata istantaneamente dalla look-up table – nessun calcolo diretto sui vettori originali durante la scansione dell'indice, il che rende PQ+ADC estremamente veloce anche su collezioni di miliardi di vettori.

Product quantization: Encoding



Note: Lossy! Different vectors may have the same encoding!

Figura 6.2: Procedura PQ: il vettore originale è diviso in m sottovettori, ciascuno codificato con l'ID del centroide più vicino nel proprio codebook.

Domanda d'esame

Descrivere la struttura dati di Seismic e le differenze rispetto a un semplice inverted index. Quali i limiti dell'inverted index tradizionale e come li risolve Seismic? [Esami D, O]

Seismic è un sistema di retrieval pensato per le **Learned Sparse Representations** (es. SPLADE): vettori sparsi ad altissima dimensionalità i cui pesi, pur essendo per la maggior parte nulli, hanno una distribuzione molto diversa da quella Zipfiana del linguaggio naturale – più “piatta” e densa nelle poche componenti attive.

Un **indice invertito semplice**, con posting list (docID, peso) compresse con VByte/PFor-Delta, soffre di due limiti critici su questi vettori appresi. Primo, **l'inefficienza del dynamic pruning**: algoritmi come WAND stimano un massimo punteggio possibile (Upper Bound) per i documenti non ancora visitati, ma con pesi appresi le stime diventano poco efficaci (bound troppo larghi), costringendo a valutare quasi tutti i documenti e annullando i benefici della

potatura. Secondo, l'**overhead di decompressione e accesso memoria**: i vettori appresi hanno molte più dimensioni attive di una query testuale classica, quindi posting list molto più lunghe da decomprimere, saturando banda di memoria e CPU.

Seismic risolve questi limiti spostando il paradigma dal “documento singolo” al “blocco di documenti”, con una struttura ibrida a tre componenti: un **indice invertito a blocchi geometricamente coesi** (le posting list di ogni dimensione sono divise in blocchi, ma – a differenza dell’ordinamento per docID crescente di un indice classico – i documenti sono raggruppati per **similarità geometrica** dei loro vettori, tramite una versione shallow di K-means); ogni blocco è equipaggiato con un **Summary Vector** (un sommario/centroide dei documenti che contiene, tipicamente le componenti non-zero più grandi, sfruttando la **Concentration of Importance**: nei vettori sparsi appresi, poche componenti contribuiscono alla maggior parte del punteggio – empiricamente il 90% del prodotto scalare si concentra in circa il 15% dei termini); un **Forward Index** mantenuto in memoria per la fase di rifinitura finale, che recupera i vettori completi solo per i documenti sopravvissuti al pruning.

Il meccanismo di pruning funziona così: prima di accedere a un blocco, si calcola il prodotto scalare tra la query e il Summary Vector del blocco; se questo punteggio stimato non supera la soglia dinamica θ dei top- k correnti, l’intero blocco viene saltato senza decomprimere né analizzare i documenti uno per uno. Poiché i documenti in un blocco sono raggruppati per similarità geometrica (non per ID), se il Summary Vector è poco rilevante è molto probabile che tutti i documenti del blocco lo siano: il pruning risulta quindi molto più robusto e preciso del pruning per ID di un indice classico. Delegando il calcolo esatto al Forward Index solo per i sopravvissuti, Seismic evita accessi casuali e decompressioni inutili, riducendo drasticamente il numero di prodotti scalari necessari e permettendo latenze nell’ordine dei millisecondi anche con modelli come SPLADE in produzione su collezioni di milioni di documenti.

Seismic: Pruning

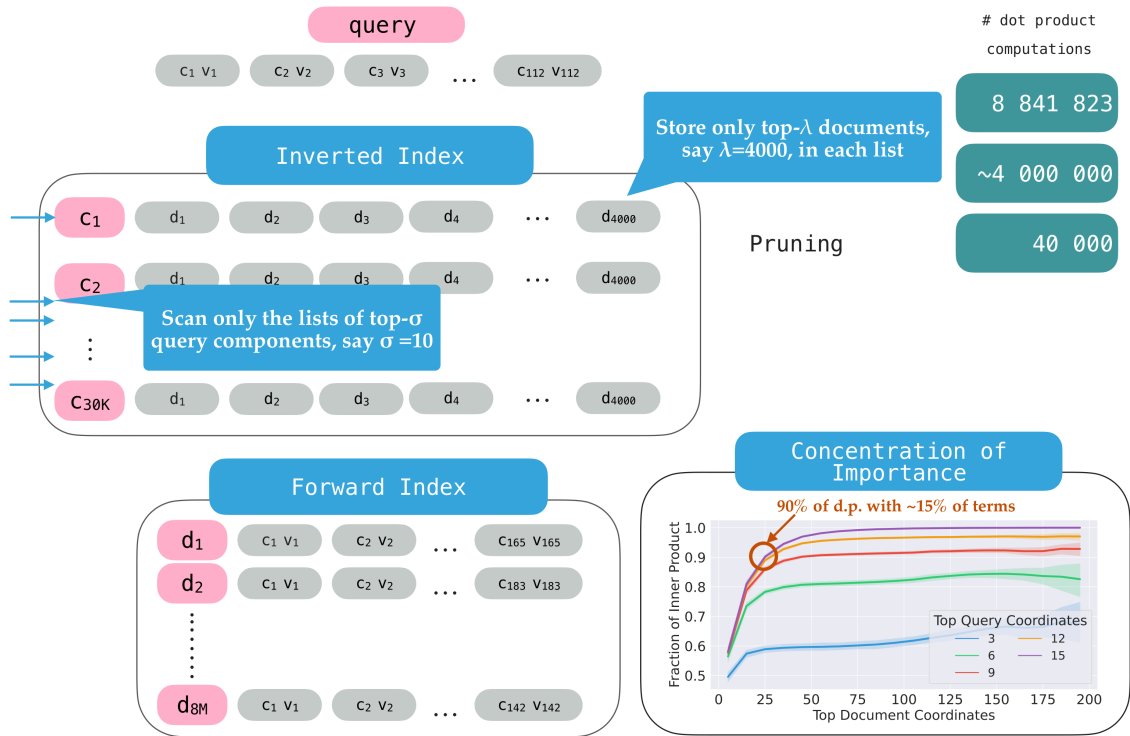


Figura 6.3: Seismic: pruning tramite Inverted Index + Forward Index, sfruttando la Concentration of Importance dei vettori sparsi appresi.

Seismic: Blocking and Summaries

- **Goal:** Skip **irrelevant** documents without computing their dot product with the query
- **Blocking:** Permute and group documents in each posting list by similarity
 - Skip an entire block if somehow we know there is no relevant document
 - Blocking with a **shallow** version of **K-Means**
 - Tens of similar documents per block
- **Summaries** to estimate the dot product of the best document in the block
 - Compute the max value for each component
 - 😊 Conservative: upper bounds true dot product
 - 😞 Too many non-zero components
 - Keep just the largest non-zero components
 - 😊 Precise even if no longer conservative
 - 😊 Reasonable space usage

d1	3.1			0.4		
d2		0.2		2.5		1.1
d3	1.5					3.5
↓						
Summary	3.1	0.2		2.5		3.5

Figura 6.4: Blocking e Summaries: un blocco viene scartato senza decomprimere i documenti se il suo vettore sommario non supera la soglia.

Capitolo 7

Materiale Supplementare (fuori dal syllabus principale)

Le seguenti domande (Esame F) vertono su argomenti classici di Information Retrieval e Web Search (spam Web, costruzione distribuita dell'indice, link analysis, strutture dati probabilistiche) che non sono coperti dalle slide del corso raccolte in *main.pdf*, ma appartengono al canone storico della materia (Manning, Raghavan, Schütze – *Introduction to Information Retrieval*) e ricorrono in alcune formulazioni d'esame. Le risposte seguenti sono fornite per completezza sulla base della letteratura standard.

Domanda d'esame

Descrivere il problema dello spamming (es. su piattaforme di hosting gratuito come Altervista) in relazione al TF-IDF.

Il TF-IDF assegna punteggio più alto a documenti che ripetono termini rari e discriminanti rispetto alla collezione: questo lo rende vulnerabile al **keyword stuffing**, la tecnica di Web spam più elementare, in cui una pagina ripete artificialmente termini di query popolari (spesso invisibili all'utente: testo bianco su sfondo bianco, font a dimensione zero, meta-tag nascosti) per gonfiare il proprio *tf* e quindi il proprio punteggio, senza offrire contenuto realmente pertinente. Piattaforme di hosting gratuito storicamente permissive (come Altervista) sono state un terreno fertile per questo fenomeno, proprio perché permettevano di creare in massa pagine a basso costo mirate a sfruttare esattamente questo punto debole del modello vettoriale puro. Le contromisure principali sviluppate dai motori di ricerca includono: penalizzare la crescita innaturale/eccessiva della densità di keyword (il BM25 stesso, con la sua saturazione della frequenza, è già più robusto del TF-IDF puro a questo tipo di abuso); affiancare al segnale testuale segnali di autorevolezza indipendenti dal contenuto della pagina, come PageRank o altri segnali di link analysis, difficili da falsificare quanto il semplice testo; usare classificatori di spam dedicati, addestrati su feature come la distribuzione statistica del testo, la presenza di testo nascosto via CSS, il cloaking (mostrare contenuto diverso a crawler e utenti) e pattern di link innaturali (link farm).

Domanda d'esame

BSBI e SPIMI: descrizione e differenze.

Entrambi sono algoritmi per costruire un indice invertito quando la collezione non entra in memoria principale, elaborando la collezione a blocchi e poi fondendo i risultati parziali (approccio *sort-based external merge*).

Il **Blocked Sort-Based Indexing (BSBI)** legge la collezione a blocchi che entrano in memoria; per ogni blocco genera tutte le coppie (**termID**, **docID**) (richiede quindi di aver già mappato

ogni termine a un identificatore intero, tipicamente tramite un dizionario costruito in una prima passata sui dati), le ordina in memoria per termID (poi per docID), scrive il blocco ordinato su disco come un mini-indice invertito, e alla fine fonde (*merge*) tutti i blocchi ordinati su disco con un merge sort esterno a k vie, producendo l'indice finale.

Il **Single-Pass In-Memory Indexing (SPIMI)** elimina la necessità di un dizionario globale di termID costruito a priori: per ogni token letto, se il termine non è ancora nel dizionario locale del blocco (una hash map termine $\rightarrow$ posting list) lo aggiunge, altrimenti accoda direttamente il docID alla posting list già esistente in memoria, evitando così la fase di ordinamento esplicito delle coppie (termID, docID) – le posting list si costruiscono incrementalmente e sono già “raggruppate per termine” per costruzione. Quando la memoria disponibile per il blocco si esaurisce, si scrive su disco il dizionario e le posting list di quel blocco (già ordinati per termine, dato che si può ordinare solo il dizionario, molto più piccolo dell'insieme delle coppie), e si riparte con un nuovo blocco vuoto; alla fine si fondono tutti i blocchi come in BSBI.

La differenza principale è quindi che BSBI richiede di ordinare esplicitamente un numero di coppie (termID, docID) pari al numero di token dell'intera collezione (costo $O(T \log T)$), mentre SPIMI costruisce l'indice di ogni blocco direttamente in forma di dizionario+posting list senza mai ordinare le coppie termine-documento, risultando generalmente più semplice, più veloce in pratica e più scalabile su vocabolari che crescono dinamicamente (non richiede di conoscere/fissare in anticipo lo spazio dei termID); per questo SPIMI è l'approccio preferito nella maggior parte dei sistemi di indicizzazione reali su larga scala.

Domanda d'esame

HITS e steady state probability: differenza con PageRank.

PageRank calcola un punteggio di autorevolezza *globale e indipendente dalla query*, definito come la distribuzione stazionaria (*steady state*) di una catena di Markov che modella un “random surfer”: con probabilità d (damping factor) l'utente segue un link uscente scelto uniformemente a caso dalla pagina corrente, con probabilità $1 - d$ salta (*teleport*) a una pagina qualsiasi del grafo scelta uniformemente. Il fattore di teleport garantisce che la catena sia irriducibile e aperiodica, quindi (per il teorema di Perron-Frobenius) esiste un'unica distribuzione stazionaria a cui la catena converge indipendentemente dallo stato iniziale: questa distribuzione, calcolata *una sola volta offline* sull'intero grafo del Web, è il PageRank di ogni pagina, ed è letteralmente la probabilità di trovare il random surfer su quella pagina dopo infiniti passi.

HITS (Hyperlink-Induced Topic Search) calcola invece due punteggi distinti e *dipendenti dalla query*: *hub* (quanto una pagina punta a buone risorse) e *authority* (quanto una pagina è una buona risorsa), su un piccolo sottografo raccolto al momento della query (root set dei risultati testuali top-ranked, espanso con le pagine collegate in entrata/uscita, il *base set*). I due punteggi si aggiornano per mutuo rinforzo iterativo: $authority(p) = \sum_{q \rightarrow p} hub(q)$ e $hub(p) = \sum_{p \rightarrow q} authority(q)$, convergendo (dopo normalizzazione) rispettivamente al principale autovettore delle matrici $A^T A$ e $A A^T$, dove A è la matrice di adiacenza del sottografo.

La differenza concettuale chiave riguardo alla “steady state probability” è quindi: il PageRank è *per definizione* una probabilità stazionaria di una catena di Markov ergodica, calcolata una volta sull'intero grafo e riusata per ogni query; gli score di HITS non sono probabilità di una passeggiata aleatoria stazionaria in questo senso stretto (non c'è damping/teleport a garantire ergodicità, e non c'è un'unica catena ma un mutuo rinforzo tra due insiemi di score su un grafo diverso per ogni query) – sono piuttosto punteggi di *eigenvector centrality* calcolati al volo, topic-dependent, e per questo più sensibili a fenomeni come il “topic drift” o la dominanza di siti molto interconnessi (mutually reinforcing relationships) all'interno del piccolo sottografo raccolto per quella query specifica.

Domanda d'esame

Bloom Filter: probabilità di errore, con dimostrazione.

Un **Bloom Filter** rappresenta un insieme di n elementi tramite un array di m bit (inizialmente tutti a 0) e k funzioni hash indipendenti $h_1, \dots, h_k$ che mappano ogni elemento uniformemente a caso su una posizione in $\{0, \dots, m-1\}$. All'inserimento di un elemento, si pongono a 1 i k bit da esso indicati; per verificare l'appartenenza di un elemento, si controlla se tutti i k bit corrispondenti sono a 1 – se anche uno solo è 0, l'elemento è certamente assente (nessun falso negativo); se tutti sono a 1, l'elemento è *probabilmente* presente (possibile falso positivo, mai falso negativo).

Dimostrazione della probabilità di falso positivo. Dopo aver inserito n elementi (ciascuno tramite k hash, quindi kn “operazioni di scrittura” totali sui bit), la probabilità che un bit specifico non sia mai stato posto a 1 da nessuna di queste kn operazioni è

$$p_0 = \left(1 - \frac{1}{m}\right)^{kn}$$

poiché ogni singola operazione ha probabilità $1/m$ di colpire quel bit specifico, e le operazioni sono indipendenti. Usando il limite notevole $\left(1 - \frac{1}{m}\right)^m \rightarrow e^{-1}$ per $m \rightarrow \infty$, si approssima

$$p_0 \approx e^{-kn/m}$$

Un **falso positivo** si verifica quando, interrogando un elemento mai inserito, tutti i suoi k bit risultano comunque a 1 per effetto di altri inserimenti. Assumendo (approssimazione standard, valida per m grande) che i bit siano impostati indipendentemente l'uno dall'altro con probabilità $1 - p_0$ ciascuno, la probabilità che tutti e k i bit dell'elemento interrogato siano a 1 è

$$f = (1 - p_0)^k \approx \left(1 - e^{-kn/m}\right)^k$$

Per trovare il numero ottimale di funzioni hash k^* che minimizza f a m e n fissati, si pone $g(k) = \ln f = k \ln (1 - e^{-kn/m})$ e si annulla la derivata rispetto a k : il minimo si ottiene per

$$k^* = \frac{m}{n} \ln 2 \approx 0.693 \frac{m}{n}$$

sostituendo k^* in f si ottiene la probabilità di falso positivo minima raggiungibile a parità di spazio m/n bit per elemento:

$$f^* \approx (0.5)^{k^*} = (0.6185)^{m/n}$$

Questo risultato è il motivo per cui i Bloom Filter sono valutati in termini di “bit per elemento” (m/n): fissato questo budget di memoria, la probabilità di falso positivo decresce esponenzialmente all'aumentare di m/n , ed esiste sempre un numero ottimo di funzioni hash k^* (non troppe, non troppo poche) che la minimizza – usarne di più di k^* satura troppo rapidamente il filtro di bit a 1 (aumentando f), usarne di meno non sfrutta a pieno lo spazio disponibile.